05_utilidad_consulta_ruc

May 9, 2026

1 05. Utilidad de Consulta RUC

1.1 Goal

Inspect one problematic RUC directly against SRI files when a match looks suspicious.

1.2 Inputs

- `01_data_ingestion_enrichment/data_SRI/`

1.3 Outputs

- Manual validation evidence for a single RUC.

[1]:
```
#   Configuración
from pathlib import Path
import re, unicodedata
import pandas as pd

try:
    from IPython.display import display
except Exception:
    def display(x): print(x)

def find_project_root(start=None):
    start = (start or Path.cwd()).resolve()
    for p in [start, *start.parents]:
        if (p / "01_data_ingestion_enrichment").is_dir() and (p /␣
 ↪"02_data_cleaning").is_dir():
            return p
    raise FileNotFoundError(f"No se pudo localizar la raíz.")

ROOT = find_project_root()

_SRI_CANDIDATES = [
    ROOT / "02_data_cleaning" / "data_SRI",
    ROOT / "01_data_ingestion_enrichment" / "data_SRI",
]
```

```
SRI_DIR = next((p for p in _SRI_CANDIDATES if p.exists() and any(p.iterdir())),␣
 ↪None)
if SRI_DIR is None:
    raise FileNotFoundError(f"No encontré data_SRI en ninguna de:␣
 ↪{_SRI_CANDIDATES}")

#   RUC a consultar (cambia aquí)
RUC_CONSULTA = "0993044008001"   # <  modifica este valor

print(f"[CONFIG] SRI_DIR : {SRI_DIR}  existe={SRI_DIR.exists()}")
print(f"[CONFIG] RUC     : {RUC_CONSULTA}")
```

```
[CONFIG] SRI_DIR : E:\TESIS
MAESTRIA\Desarrollo_clustering_maestria\02_data_cleaning\data_SRI  existe=True
[CONFIG] RUC     : 0993044008001
```

1.4 Migrate Here From Source Notebook

- Single RUC lookup utility.

1.4.1 Suggested Source Cells

- Code cell: 29

[2]:
```
#   Búsqueda del RUC en los archivos SRI

# Re-verificar SRI_DIR por si el kernel tiene variable desactualizada
if not SRI_DIR.exists():
    _SRI_CANDIDATES = [
        ROOT / "02_data_cleaning" / "data_SRI",
        ROOT / "01_data_ingestion_enrichment" / "data_SRI",
    ]
    SRI_DIR = next((p for p in _SRI_CANDIDATES if p.exists() and any(p.
 ↪iterdir())), None)
    if SRI_DIR is None:
        raise FileNotFoundError(f"No encontré data_SRI en ninguna de:␣
 ↪{_SRI_CANDIDATES}")
    print(f"[INFO] SRI_DIR reasignado: {SRI_DIR}")

ruc_target = re.sub(r"\D+", "", str(RUC_CONSULTA).strip())
if len(ruc_target) != 13:
    raise ValueError(f"RUC inválido (debe tener 13 dígitos): {RUC_CONSULTA}")

csv_files = sorted([p for p in SRI_DIR.iterdir() if p.is_file() and p.suffix.
 ↪lower() == ".csv"])
if not csv_files:
    raise FileNotFoundError(f"No hay CSV en: {SRI_DIR}")
```

```
def _sniff_sep(fp: Path) -> str:
    try: head = fp.open("rb").read(4096)
    except Exception: return ","
    counts = {"|": head.count(b"|"), ";": head.count(b";"), ",": head.
 ↪count(b","), "\t": head.count(b"\t")}
    sep = max(counts, key=counts.get)
    return sep if counts.get(sep, 0) > 0 else ","

def _norm_col(col: str) -> str:
    s = str(col or "").strip().upper()
    s = "".join(c for c in unicodedata.normalize("NFKD", s) if not unicodedata.
 ↪combining(c))
    return re.sub(r"[^A-Z0-9]+", "", s)

def _pick_sri_cols(fp: Path):
    sep = _sniff_sep(fp)
    for enc in ["utf-8-sig","utf-8","cp1252","latin1"]:
        try:
            hdr = pd.read_csv(fp, sep=sep, encoding=enc, dtype=str, nrows=0,␣
 ↪on_bad_lines="skip")
            n2o = {_norm_col(c): c for c in hdr.columns}
            col_ruc   = next((n2o[a] for a in ["NUMERORUC","NUMERODERUC","RUC"]␣
 ↪if a in n2o), None)
            col_razon = next((n2o[a] for a in ["RAZONSOCIAL","RAZONSOC"] if a␣
 ↪in n2o), None)
            col_fan   = next((n2o[a] for a in␣
 ↪["NOMBREFANTASIACOMERCIAL","NOMBREFANTASIA","NOMBRECOMERCIAL"] if a in n2o),␣
 ↪None)
            if col_ruc:
                usecols = [c for c in [col_ruc, col_razon, col_fan] if c]
                return sep, enc, col_ruc, col_razon, col_fan, usecols
        except Exception: continue
    raise RuntimeError(f"No pude detectar columnas en {fp.name}")

results = []
CHUNKSIZE = 200_000

for i, fp in enumerate(csv_files, 1):
    try: sep, enc, col_ruc, col_razon, col_fan, usecols = _pick_sri_cols(fp)
    except Exception as e: print(f"[{i}/{len(csv_files)}] {fp.name}: saltado␣
 ↪({e})"); continue
    hits = 0
    try:
        for chunk in pd.read_csv(fp, sep=sep, encoding=enc, dtype=str,␣
 ↪usecols=usecols,
```

```
                                    chunksize=CHUNKSIZE, on_bad_lines="skip",␣
↪low_memory=True):
            if chunk is None or chunk.empty: continue
            chunk[col_ruc] = chunk[col_ruc].astype("string").fillna("").map(
                lambda x: re.sub(r"\D+", "", str(x)))
            hit = chunk[chunk[col_ruc] == ruc_target]
            if hit.empty: continue
            hits += len(hit)
            for _, row in hit.iterrows():
                results.append({
                    "archivo": fp.name,
                    "NUMERO_RUC":    row.get(col_ruc, ""),
                    "RAZON_SOCIAL":  row.get(col_razon, "") if col_razon else␣
↪"",
                    "NOMBRE_FANTASIA": row.get(col_fan, "") if col_fan else "",
                })
    except Exception as e: print(f"[{i}/{len(csv_files)}] {fp.name}: ERROR ->␣
↪{e}"); continue
    if hits > 0: print(f"[{i}/{len(csv_files)}] {fp.name}: {hits} filas␣
↪encontradas")

print(f"\n   RESULTADO para RUC {ruc_target}   ")
print(f"Total registros encontrados: {len(results)}")

df_ruc = pd.DataFrame(results)
if df_ruc.empty:
    print("No se encontraron registros para ese RUC en data_SRI.")
else:
    display(df_ruc)
```

```
[12/26] SRI_RUC_Guayas.csv: 1 filas encontradas

   RESULTADO para RUC 0993044008001
Total registros encontrados: 1
```

	archivo	NUMERO_RUC	RAZON_SOCIAL	NOMBRE_FANTASIA
0	SRI_RUC_Guayas.csv	0993044008001	CLG S.A.	NaN